UYGULAMALI ATÖLYE

Ajanları Dış Dünyaya Bağlamak

Model Context Protocol (MCP), LangGraph ve adım adım büyüyen bir Kampüs Asistanı

🧠 LLM ve ajanlar 🔌 MCP 🕸️ LangGraph 🐘 PostgreSQL 📡 Uzak server

TEORİ

Bir LLM tek başına neden yetmez?

Büyük dil modeli (LLM), metne bakıp **bir sonraki parçayı tahmin eden** bir modeldir. Çok şey "bilir", ama:

- Bilgisi **eğitim verisiyle sınırlıdır**; dünü bilmez.
- **Senin verine erişemez:** ders programın, ödevlerin, şirketinin veritabanı.
- Dünyada **bir şey yapamaz:** dosya yazamaz, e-posta atamaz, kayıt ekleyemez.
- Bilmediğinde bazen **uydurur** (halüsinasyon).

👤 Kullanıcı

Yarın yemekhanede ne var?

🤖 Sadece LLM

"Yarın mercimek çorbası ve tavuk sote var!"

↑ Kendinden emin, ama tamamen uydurma.

TEORİ

Çözüm: LLM'e araç vermek

LLM aracı **kendisi çalıştırmaz**. Hangi aracın hangi parametrelerle çağrılacağını **önerir**; aracı bizim programımız çalıştırır ve sonucu LLM'e geri verir.

1 · Soru
Kullanıcı sorar. Program, soruyla birlikte **araç listesini** de LLM'e gönderir.

→

2 · Karar
LLM metin yerine bir **araç çağrısı** üretir:
`yemekhane_menusu(gun="yarın")`

→

3 · Çalıştırma
Program gerçek fonksiyonu çalıştırır: dosyayı okur, veritabanına sorar...

→

4 · Sonuç
Sonuç (“Pazar: Yemekhane kapalı”) LLM'e geri gönderilir.

→

5 · Cevap
LLM, **gerçek veriye dayanarak** cevabı yazar.

> Kontrol her zaman programda kalır: LLM sadece **ne yapılmasını istediğini söyler**. Bu ayrım, güvenlik açısından çok önemli.

TEORİ

Ajan = LLM + araçlar + döngü



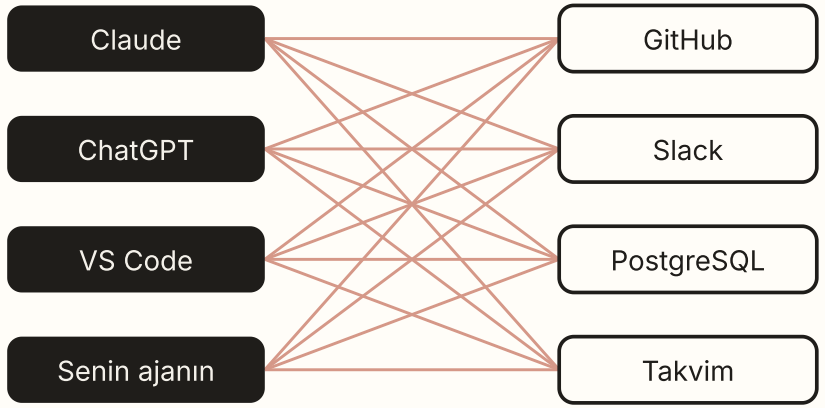
Bir **ajan**, LLM'in araçları **kendi kararıyla** ve gerektiği kadar **tekrar tekrar** kullandığı bir döngüdür (**ReAct**: Reason + Act).

- Klasik programda kararları **if/else** ile biz veririz.
- Ajanda kararı **LLM** verir: hangi araç, hangi sırayla, kaç kez.
- Araç hata verirse ajan hatayı okuyup **kendini düzeltebilir**.
- Bedeli: daha az öngörülebilirlik. Bu yüzden **sınırlar ve denetim** şart.

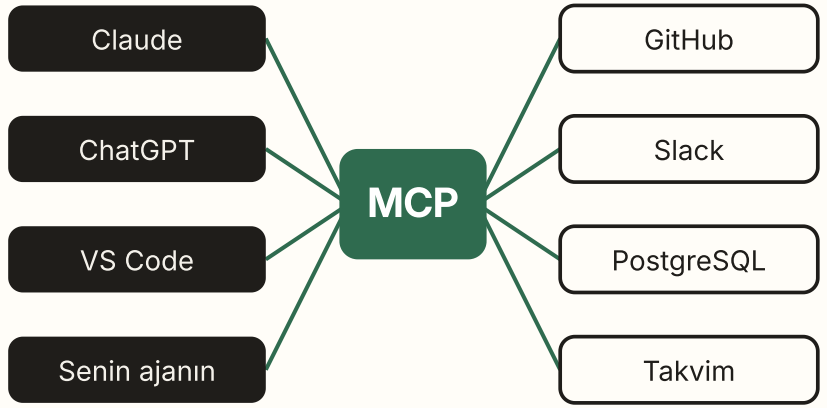
TEORİ

MCP: araçlar için ortak bir standart

MCP'den önce: 4 × 4 = 16 ayrı entegrasyon



MCP ile: 4 + 4 = 8 standart bağlantı



Bir aracı **bir kez** MCP server olarak yazarsın; MCP destekleyen **her** uygulama onu kullanabilir. Sık yapılan benzetme: **"Yapay zekâ için USB-C."**

Kasım 2024
Anthropic açık standart olarak duyurdu.

Aralık 2025
Linux Foundation'daki Agentic AI Foundation'a devredildi.

Bugün
ChatGPT, Gemini, Copilot, VS Code, Cursor, Claude; 10.000+ server.

TEORİ

Mimari: Host, Client, Server

Host

Kullanıcının etkileştiği uygulama, LLM'i yönetir. Claude Desktop, VS Code, bizim `ajan.py`

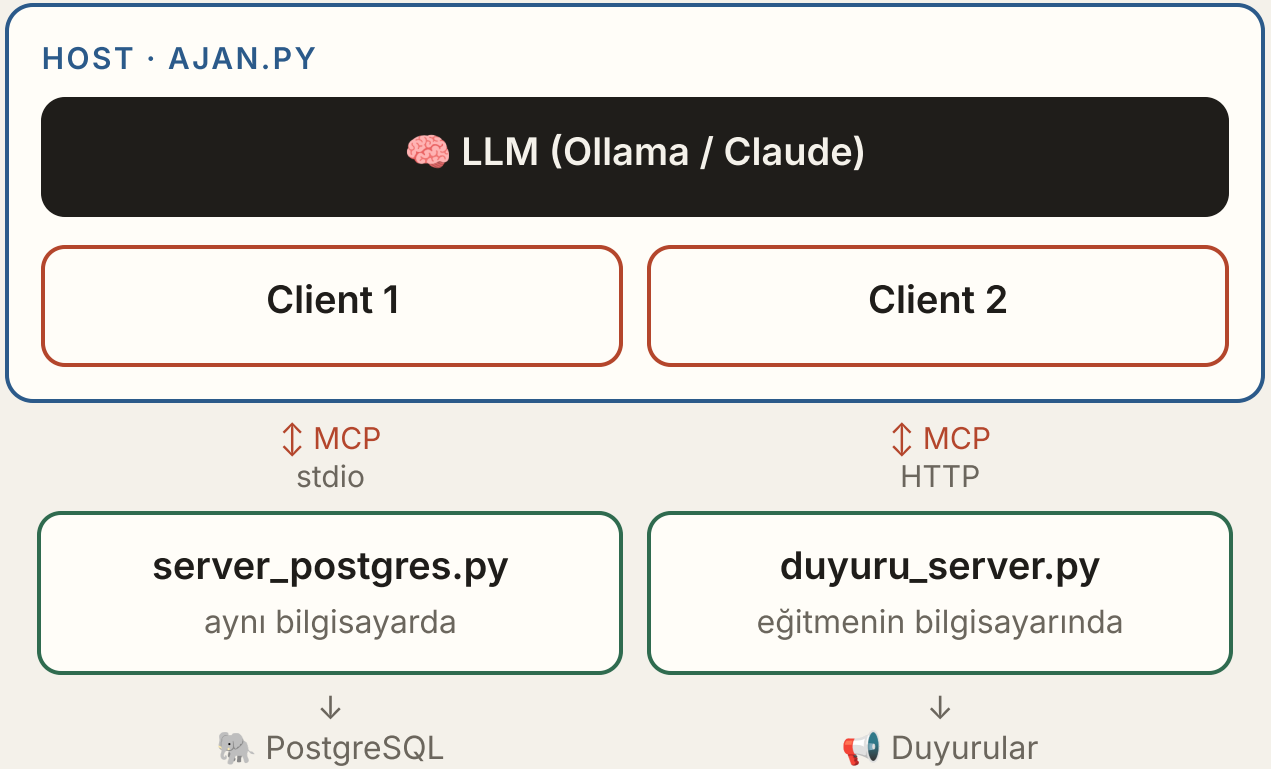
Client

Host'un içinde, **her server için bir tane** açılan bağlantı.

Server

Araçları ve verileri sunan program. **Yerelde** veya **uzakta** çalışır.

Server üç şey sunabilir: **Tools** (araçlar, bugünün konusu), **Resources** (okunabilir veriler), **Prompts** (hazır şablonlar).



TEORİ

Protokol: satır satır JSON-RPC

1. `initialize` : İstemci ve server el sıkışır.
2. `tools/list` : İstemci araçların adlarını, **açıklamalarını** ve şemalarını öğrenir.
3. `tools/call` : İstemci bir aracı çağırır, server sonucu döner.

TAŞIMA	NASIL?
stdio	İstemci server'ı alt süreç olarak başlatır; aynı bilgisayar.
Streamable HTTP	Server bağımsız bir web servisi; istemci ağ üzerinden bağlanır.

→ İSTEMCİ (AJAN)

```
{"jsonrpc": "2.0", "id": 3,
 "method": "tools/call",
 "params": {"name": "yemekhane_menusu",
            "arguments": {"gun": "salı"}}}
```

← SERVER

```
{"jsonrpc": "2.0", "id": 3,
 "result": {
   "content": [{"type": "text",
     "text": "Salı menüsü: Ezogelin çorbası, ..."}],
   "isError": false}}
```

> **LLM kodu görmez**, sadece aracın `description`'ını okur. İyi açıklama = doğru araç seçimi.

UYGULAMA

Kampüs Asistanı: dört adımda bir ajan

1

İlk MCP server

`server.py` · JSON dosyaları

Protokolü **sıfırdan**, kütüphanesiz yazıyoruz.

2

LangGraph ajanı

`ajan.py` · Ollama

Kararı LLM'e bırakıyoruz: graf, araçlar, hafıza.

3

Veritabanı

`server_postgres.py` · Docker

Aynı ajan, farklı server. SQL yazan ajan ve güvenlik.

4

Uzak server

`duyuru_server.py` · HTTP

Ağ üzerinden bağlantı ve iki server'ı birleştiren ajan.

> Her bölümün sonunda **canlı demo** var. Tüm kodlar ve adım adım rehber: `kampus-asistani/README.md`

Sıfırdan bir MCP server

Kütüphanesiz, yalnızca Python: **araç fonksiyonları** + **araç tanımları** (`ARACLAR`) + **satır satır JSON okuyan bir döngü**.

Kural 1: stdout sadece protokol içindir

Server'daki bir `print()` istemciyi bozar. Loglar **stderr**'e gider.

Kural 2: Araç hatası da bir cevaptır

Hatalı girdide `"isError": true` dönülür; ajan hatayı okuyup **düzeltebilir**.

CANLI DEMO `python server.py`

```
python server.py

{"jsonrpc":"2.0","id":1,"method":"initialize","params":{}}

{"jsonrpc":"2.0","id":2,"method":"tools/list"}

{"jsonrpc":"2.0","id":3,"method":"tools/call",
 "params":{"name":"yemekhane_menusu",
           "arguments":{"gun":"salı"}}}

{"jsonrpc":"2.0","id":4,"method":"tools/call",
 "params":{"name":"odev_ekle","arguments":{
   "ders":"Algoritmalar","aciklama":"Deneme",
   "son_tarih":"yarın akşam"}}}   # → isError
```

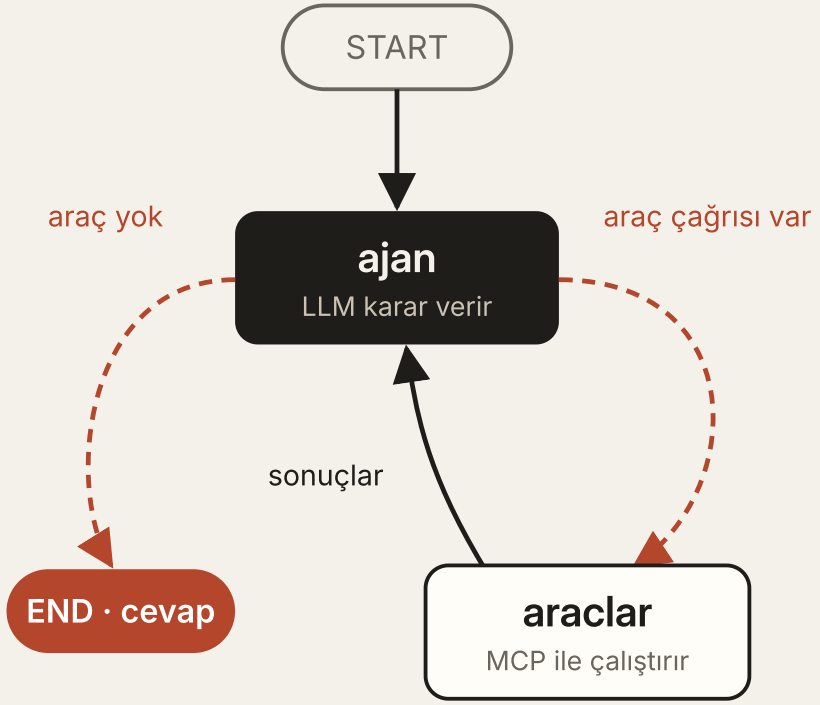
LangGraph: ajanı bir graf olarak kurmak

- **Durum (State):** Graf boyunca taşınan veri. Bizde: mesaj geçmişi.
- **Düğüm (Node):** Durumu okuyup güncelleyen bir fonksiyon.
- **Koşullu kenar:** "Araç çağrısı var mı?" sorusuna göre yol seçimi.
- **Checkpointer:** Durumu saklar; sohbet hafızası sağlar ("Onu tamamla").

MCP ARAÇLARI → LANGCHAİN ARAÇLARI

```
async with istemci.session("kampus") as oturum:
    araclar = await load_mcp_tools(oturum)
graf = graf_olustur(model, araclar)
```

Ajan kodunda **tek bir araç bile** elle tanımlanmadı; hepsi server'dan geliyor.



BÖLÜM 2

Ajan iş başında

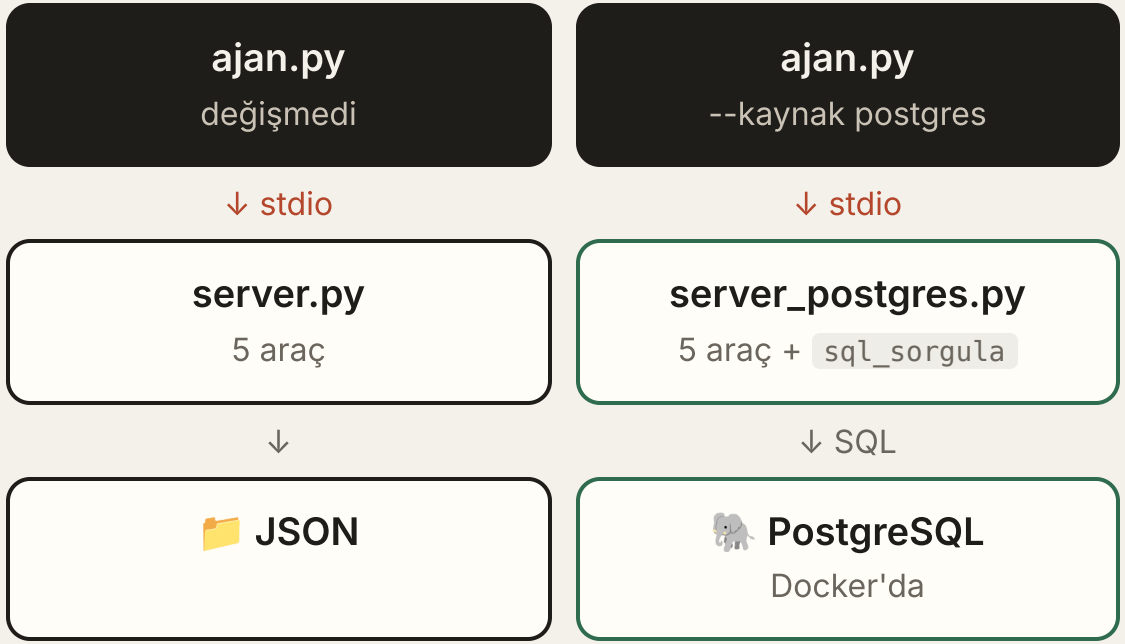
SORU	NEYİ GÖSTERİR?
“Salı derslerim ve yemekte ne var?”	Tek seferde iki araç
“25 Ekim'e graf ödevi ekle”	Hata alır, tarihi düzeltip tekrar dener
“Onu tamamla”	Hafıza
“Bana bir fıkra anlat”	Araç gerekmeyince doğrudan cevap

CANLI DEMO `python ajan.py --protokol`

```
👤 Sen: Algoritmalar için 25 Ekim'e graf ödevi ekle
🧠 [ajan] → odev_ekle({'son_tarih': '25 Ekim', ...})
🔧 Hata: '25 Ekim' geçerli bir tarih değil.
   YYYY-AA-GG biçiminde olmalı.
🧠 [ajan] → odev_ekle({'son_tarih': '2026-10-25', ...})
🔧 Ödev #2 eklendi.
🤖 Asistan: Graf ödevini 25 Ekim'e ekledim.

👤 Sen: Onu tamamla
🧠 [ajan] → odev_tamamla({'id': 2})
🔧 Ödev #2 tamamlandı olarak işaretlendi.
```

Aynı ajan, farklı server



Araç **adları aynı** olduğu için ajan farkı anlamaz bile.

RESMİ SDK (FASTMCP) İLE BİR ARAÇ

```python
mcp = FastMCP("kampus-postgres")

@mcp.tool()
def ders_programi(gun: str = "bugün") -> str:
    """Öğrencinin ... getirir."""
    satirlar = sorgula(
        "SELECT ... WHERE p.gun = %s",
        (gun_coz(gun),))
    ...
```

Fonksiyon adı → araç adı · docstring → açıklama · tip ipuçları → şema. Protokol döngüsünü SDK yönetir.

CANLI DEMO `docker compose up -d` · `python ajan.py --kaynak postgres --protokol`

Kendi SQL'ini yazan ajana güvenme

Hazır araç olmayan sorularda (“Hangi derste kaç ödevim var?”) ajan `sql_sorgula` ile **kendi SQL'ini yazar**. Peki biri ajanı “tüm ödevleri sil” diye kandırırsa (**prompt injection**)?

KATMAN	NE YAPAR?
1. Kod kontrolü	`SELECT` / `WITH` ile başlamalı, tek sorgu
2. Salt okunur kullanıcı	`ajan_okuyucu` : sadece `SELECT` yetkisi
3. READ ONLY bağlantı	Salt okunur oturum, 5 sn süre sınırı

1. katman neden tek başına yetmez?

```
WITH s AS (
  DELETE FROM odevler RETURNING *
)
SELECT count(*) FROM s
```

`WITH` ile başlıyor, kontrolü geçiyor; ama bir silme işlemi. **2. ve 3. katman durdurur.**

> **En az yetki ilkesi:** En güçlü savunma, veritabanı yetkileridir.

Uzak server: ajan duyuruyu nereden öğreniyor?

① Eğitmen
Panelden "Pazartesi Algoritmalar iptal" yayınlar. **Hiçbir ajan habersiz.**

→

② Öğrenci
"Pazartesi hangi derslerim var?"

→

③ LLM karar verir
Açıklama: "ders programı sorulunca **MUTLAKA** duyuruları da kontrol et"

→

④ İki server
`ders_programi` → yerel `duyurulari_getir` → HTTP

→

⑤ Cevap
"Algoritmalar iptal; sadece 13:00'te Veritabanı var."

AJANIN TEK FARKI: BAĞLANTI AYARI

```
serverlar = {
  "kampus": {"transport": "stdio", ...},
  "duyuru": {"transport": "streamable_http",
             "url": "http://<ip>:8000/mcp"},
}
```

Çekme (pull): Ajan duyuruyu ancak **ihtiyaç duyduğunda** çeker. Kodda "duyuruya bak" diye bir `if` yok; kararı **açıklama** verdiriyor.

Eğitmen paneli: `localhost:8000/panel`. Duyuru, gelen sorular, canlı anket.

CANLI DEMO `python duyuru_server.py` · `python ajan.py --duyuru http://<ip>:8000/mcp --ad "Adın"`

ÖZET

Akılda kalacaklar

1. LLM araçları **önerir**, program **çalıştırır**.
2. LLM kodu değil, **açıklamayı** görür.
3. Ajan = LLM + araçlar + **döngü**.
4. Server değişir, **ajan aynı kalır**.
5. MCP: **bir kez yaz**, her yerde kullan.
6. Ajan yerel ve uzak **birden fazla server'ı** birleştirir.
7. MCP özünde **satır satır JSON-RPC**'dir.
8. Ajanlara **güven değil, sınır** ver.

Teşekkürler · Sorular?

Alıştırmalar ve adım adım rehber: `README.md`

"Hocaya sor: ..."